Cognitive Modeling

Complete Lecture Notes

Week 6: Evaluating Models I: Goodness of Fit, Overfitting and Cross-Validation

Contents

1 Evaluation of Cognitive Models 1

1.1 Week 6: Evaluating Models I: Goodness of Fit, Overfitting and Cross-Validation . . . 1

1.1.1 Key terms . . . 1

1.1.2 Learning outcomes . . . 2

1.1.3 Motivation & big picture . . . 3

1.1.4 Lecture notes . . . 3

1.1.5 Worked example . . . 8

1.1.6 Lab (2 hours) . . . 9

1.1.7 Connection to AI & LLMs . . . 13

1.1.8 Common mistakes & misconceptions . . . 14

1.1.9 Check your understanding . . . 14

1.1.10 Assignment . . . 15

1.1.11 Further reading . . . 16

Chapter 1

Evaluation of Cognitive Models

1.1 Week 6: Evaluating Models I: Goodness of Fit, Overfitting and Cross-Validation

Chapter 3: Evaluation of cognitive models · Lecture 1 h · Lab 2 h Running data: the Week 2–5 forgetting data, now with four candidate models: exponential ($k = 2$), power ($k = 2$), exponential-with-asymptote ($k = 3$) and two-trace ($k = 4$).

1.1.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
goodness of fit	نیکویی برازش	اینکه مدل داده‌ی موجود را چقدر خوب توصیف می‌کند
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
overfitting	بیش‌برازش (overfitting)	برازش نویز نمونه به‌جای الگوی واقعی
forgetting curve	منحنی فراموشی	کاهش یادآوری با گذشت زمان
exponential / power function	توانی / تابع نمایی	دو شکل رایج برای منحنی فراموشی و یادگیری
trace plot	نمودار trace	مقدار نمونه‌ها در برابر شماره‌ی تکرار
model flexibility / complexity	پیچیدگی مدل / انعطاف‌پذیری	دامنه‌ی الگوهایی که مدل می‌تواند تولید کند

English	فارسی	توضیح کوتاه
training / test data	آزمون / داده‌ی آموزش	داده‌ی کنارگذاشته برای / داده‌ی برازش سنجش
cross-validation (CV)	اعتبارسنجی متقابل (cross-validation)	برازش روی بخشی از داده و آزمون روی بخش کنارگذاشته
log-likelihood / NLL	منفی آن / لگاریتم درست‌نمایی	مقداری که بهینه‌سازها کمینه یا بیشینه می‌کنند
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
generalization	تعمیم	پیش‌بینی خوب داده‌ی جدید
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
SSE / RMSD	ریشه‌ی میانگین / مجموع مربعات خطا مربع انحراف	معیارهای فاصله‌ی داده و پیش‌بینی
noise	نویز	تغییرات تصادفی در شواهد یا در داده
response bias	سوگیری پاسخ	تمایل به یک پاسخ، مستقل از محرک
criterion (c)	ملاک	آستانه‌ی تصمیم؛ نشان‌دهنده‌ی سوگیری پاسخ
bias–variance trade-off	بده‌بستان سوگیری و واریانس	مدل ساده سوگیر، مدل منعطف پرواریانس
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
data leakage	نشت داده	رسیدن اطلاعات داده‌ی آزمون به برازش
AIC / BIC	بیزی / معیار اطلاعاتی آکائیکه	برازش به‌علاوه‌ی جریمه برای تعداد پارامتر
benchmark	benchmark (مجموعه‌آزمون استاندارد)	مجموعه سؤال برای مقایسه‌ی مدل‌های هوش مصنوعی
benchmark contamination	آلودگی benchmark	بودن سؤال‌های آزمون در داده‌ی آموزش مدل
parameter recovery	بازیابی پارامتر	آیا برازش، پارامترهای معلوم شبیه‌سازی را برمی‌گرداند

1.1.2 Learning outcomes

By the end of this week, students will be able to:

1. Explain why a good fit is necessary but not sufficient evidence for a model, using the concepts of flexibility and overfitting.
2. Demonstrate overfitting by simulation: training error falls monotonically with model complexity while prediction error does not.

3. Implement hold-out, K-fold and leave-one-out cross-validation with an out-of-sample log-likelihood score, keeping the training and test data properly separated.
4. Choose what to hold out (trials, conditions, participants) according to the generalization claim being made.
5. Interpret cross-validation results, including their uncertainty, to compare cognitive models.

1.1.3 Motivation & big picture

In Week 3, Exercise 4, the three-parameter exponential-with-asymptote model fitted the Week 2 data *better* than the power model (SSE ≈ 0.020 vs. 0.026), although the data were generated by the power model. In Week 3, Part D3, a four-parameter two-trace model did just as well. If we chose models by fit, we would choose the wrong one, and we would keep adding parameters forever.

The problem is not specific to forgetting curves. A flexible enough model can fit any data, including the noise. Roberts and Pashler famously asked how persuasive a good fit really is (see Further reading). Their answer: not very, unless we also know what the model could *not* have fitted.

The way out is to remember why we fit models. We want to capture the *process* that generated the data, and a model that has captured the process should predict new data from that process. Noise, by definition, does not repeat. So the question changes from "how well does the model fit these data?" to "how well does it predict data it has not seen?" Cross-validation answers this question directly with data we already have.

Where is this used?

- Choosing between theories. Whenever two models of memory, decision or learning are compared, a good fit alone is not convincing; reviewers ask for cross-validation or a measure that penalizes flexibility.
- Prediction in clinical psychology and psychiatry: models that predict relapse, treatment response or diagnosis must be validated on *new patients*, not on the patients they were fitted to. Many published predictors fail exactly this test.
- Machine learning: train/validation/test splits and k-fold cross-validation are the standard way to choose models and hyperparameters; overfitting is the central danger.
- Evaluating AI systems: benchmark scores are only meaningful if the test items were not in the training data (see the AI & LLMs section).

1.1.4 Lecture notes

In plain words A model with enough knobs can pass through every data point, including the random noise. That does not make it a good model: the noise will be different next time, so a model

that copied the noise will predict *new* data badly. This is overfitting. The fix is to judge models by how well they predict data they have not seen. Cross-validation does this with the data we already have: hide part of the data, fit the model on the rest, predict the hidden part, and repeat until every part has been hidden once. The model with the best predictions on hidden data wins. The week's main message: a better fit to the *same* data is not evidence for a better model; better *prediction* of new data is.

Suggested timing (60 min): measures of fit and their limits 10 · overfitting and flexibility 15 · the logic of generalization 5 · cross-validation 20 · what to hold out 10.

1. Measures of goodness of fit

Measure	Definition	Scale	Remarks
RMSD	$\sqrt{\text{SSE}/N}$	Data units	Easy to read; assumes equal weighting
R^2	$1 - \text{SSE}/\sum_i (y_i - \bar{y})^2$	0–1 (for linear models)	Popular but treacherous for nonlinear models: can be negative, rewards flexibility, and is dominated by large between-condition variance (fitting a monotone decline earns a high R^2 almost automatically)
Log-likelihood $\ell(\hat{\boldsymbol{\theta}})$, NLL	Week 4	Nats	The natural measure for likelihood-based models
Deviance G^2	$2(\ell_{\text{sat}} - \ell)$	$\approx \chi^2$	Absolute test of misfit (Week 4)

All of these measure descriptive adequacy: how well the model reproduces the data it was fitted to. None of them accounts for how easily the model *could* have produced those data.

2. Overfitting and flexibility

Write each observation as signal + noise: $y_i = g(x_i) + \varepsilon_i$, where g is the true process. A fitted model tries to reproduce y_i, but only the g part will be there again in new data.

- A model that is too simple misses part of g: systematic misfit, or underfitting (high bias).
- A model that is too flexible bends to follow the particular ε_i of this sample: overfitting (high variance). Its fit to the sample improves; its predictions for new data get worse.

As complexity increases, training error always goes down, while expected prediction error first decreases and then increases: a U-shape (lab Part A).

Flexibility is more than the number of parameters. Two models with the same k can differ greatly in flexibility, i.e., in the range of data patterns they can produce. A parameter that enters as an exponent or a shift usually buys more flexibility than one that scales the curve. The two-trace model can mimic exponential, power-like and asymptotic curves, which is exactly why its good fit is unimpressive. Information criteria (Week 7) count parameters; cross-validation and Bayes factors (Week 10) respond to actual flexibility.

> Advanced: the bias–variance decomposition. For squared error, the expected prediction error at a point x (over new datasets and new noise) decomposes as
>
> $$\mathbb{E}\big[(y_{\text{new}} - \hat{f}(x))^2\big] = \underbrace{\sigma^2}_{\text{irreducible noise}} + \underbrace{\big(g(x) - \mathbb{E}[\hat{f}(x)]\big)^2}_{\text{bias}^2} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{variance}}.$$
>
> Adding parameters lowers bias but raises variance. The best model for prediction balances the two, and it is not necessarily the true model when data are scarce. This decomposition is treated in depth in Azimifar & Ahmadi (see Further reading).

Reading the math The bias–variance decomposition (Advanced box) splits the average squared prediction error into three parts:

- σ^2, irreducible noise: even the true process cannot predict the random part of new data. No model can go below this.
- bias$^2 = (g(x) - \mathbb{E}[\hat{f}(x)])^2$: how far the model's *average* prediction (averaged over many possible datasets) is from the truth. Too-simple models have large bias.
- variance $= \text{Var}[\hat{f}(x)]$: how much the fitted curve jumps around from one dataset to the next. Too-flexible models have large variance, because they chase each dataset's noise.

Adding parameters moves error from the bias term into the variance term. The best predictor sits where their sum is smallest, which is why a slightly wrong but simple model can predict better than the true but complex one when data are scarce.

3. Generalization as the criterion

We prefer the model that best predicts new data from the same process. Three kinds of "new" correspond to three scientific claims:

New data are…	Claim being tested	Hold out…
New trials of the same participant in the same conditions	The model captures this person's process	Random trials or items
New conditions (e.g., longer delays, new stimuli)	The model captures the mechanism, so it can extrapolate	Whole conditions (the *generalization criterion* approach)
New participants	The model captures what is common to people	Whole participants

Always state which kind of generalization your evaluation targets.

4. Cross-validation (CV)

Hold-out validation. Split the data into a training set (fit) and a test set (evaluate). Simple, but wasteful and noisy: the result depends on one particular split.

K-fold CV. Split the data into K roughly equal folds. For each fold $k = 1, \ldots, K$: 1. Fit the model to all data except fold k, obtaining $\hat{\boldsymbol{\theta}}_{(-k)}$. 2. Score its predictions for fold k.

Sum the scores over folds. Every observation is used once for testing, and the model is never tested on data it was trained on. Typical values are $K = 5$ or 10. Leave-one-out (LOO) is the special case where every observation is its own fold.

The score. For likelihood-based models, use the out-of-sample log-likelihood (log predictive score):

$$\text{CV-NLL} = -\sum_{k=1}^{K} \sum_{i \in \text{fold } k} \ln p\big(y_i \mid \hat{\boldsymbol{\theta}}_{(-k)}, \mathcal{M}\big).$$

Lower is better. For Gaussian models, out-of-sample squared error is an equivalent choice.

Rules for honest CV: 1. Everything that is learned from data, including parameter estimates, starting-value choices from grids and data-driven transformations, must be done *inside* each training fold. Otherwise information "leaks" from the test set. 2. Split along the right unit (§3). For trial-level CV with binomial counts, split the items within each condition, so that every training fold still contains every condition. 3. CV estimates are noisy. Repeat with several random fold assignments and report the mean and spread, or the SE of the *difference* between models across folds. 4. With the item-level split, CV compares models fitted to $(K-1)/K$ of the data, so it slightly penalizes models that need a lot of data.

> Advanced: LOO, AIC and Bayes. For regular models and large N, leave-one-out CV with the log score and AIC (Week 7) choose the same model asymptotically. In Bayesian modeling, an efficient approximation to LOO (PSIS-LOO) is computed from posterior samples. You will meet it with ArviZ in Week 13.

Reading the math CV-NLL $= -\sum_k \sum_{i \in \text{fold } k} \ln p(y_i \mid \hat{\boldsymbol{\theta}}_{(-k)}, \mathcal{M})$:

- $\hat{\boldsymbol{\theta}}_{(-k)}$: the parameters estimated without fold k (the minus sign in the subscript means "leaving out").
- $\ln p(y_i \mid \hat{\boldsymbol{\theta}}_{(-k)})$: how probable the model, trained without them, finds the held-out observations. A model that predicted them well gives them high probability.
- The double sum adds this over all folds and all held-out points; the minus sign makes it a quantity to *minimize*, like the NLL of Week 4. It is the same NLL, but each observation is scored by a model that never saw it.

5. Interpreting a model comparison

- Differences in CV-NLL are in nats. As a rough guide, differences smaller than about 1–2 nats, or smaller than twice their SE across folds or repetitions, should not be over-interpreted.
- A model that wins CV is the best predictor among those compared, for this kind of generalization and this amount of data. It is not "the true model". With more data, more complex models often start to win, because their variance shrinks.
- Combine CV with the qualitative checks you already know: residual patterns, parameter plausibility, and the model's interpretability.

For an engineer

- Overfitting is the familiar problem of fitting a high-order polynomial or a filter with too many taps to a short, noisy record: the training error goes to zero and the prediction error explodes.
- The bias–variance trade-off is the same one you meet when choosing a model order (AR order, number of filter taps) or a smoothing window: too few degrees of freedom gives bias, too many gives variance.
- Cross-validation is the data-driven version of model-order selection. AIC/MDL-type criteria (Week 7) are the analytic shortcuts you may know from AR model order selection.
- Training on the test set is like tuning a receiver on the same noise realization you later measure its BER on: the reported performance is optimistic.
- Nats are the natural-log version of bits: a difference of 1 nat is a likelihood ratio of $e \approx 2.7$.

For the instructor

- Start with the polynomial worked example on the board. Four points, four models. Let students pick the best model by training SSE (they pick the cubic), then ask what it predicts at 16 days (-8.5). The laugh is the lesson.
- Do leave-one-out by hand for one point (cover the 8-day point with your hand, refit the line by eye, predict). Students then understand what the code in the lab does.
- Insist on "what is held out". Write the three kinds of "new" data (Section 3) on the board; most mistakes in student projects come from holding out the wrong thing.
- Typical questions.

- *"If the true model overfits less, why can a wrong model win CV?"* With little data, a simpler wrong model can have lower variance. CV picks the best *predictor*, not the true model.
- *"How many folds?"* 5 or 10 is standard; LOO for very small datasets.
- *"Is $R^2 = 0.98$ good?"* For a decreasing curve, almost any monotone model gets a high R^2; it says little.

1.1.5 Worked example

Data. Four recall proportions $y = (0.62, 0.50, 0.47, 0.27)$ at $t = (1, 2, 4, 8)$ days. We compare polynomials of increasing degree as "models".

Step 1: fit to all the data

Model	k	Fitted equation	Training SSE
Constant	1	$\hat{y} = 0.465$	0.0633
Linear	2	$\hat{y} = 0.637 - 0.046\,t$	0.0032
Quadratic	3	$\hat{y} = 0.647 - 0.053\,t + 0.0008\,t^2$	0.0031
Cubic	4	passes exactly through all 4 points	0.0000

By training SSE, the cubic is "perfect". But ask it to predict recall at $t = 16$ days: it predicts -8.5, i.e., *minus 850% recall.* The linear model predicts -0.10 (also impossible, but less absurd), and the constant predicts 0.47.

Step 2: leave-one-out CV

Leave out each point in turn, refit on the remaining three, and predict the left-out point.

Left-out t	Observed	Constant predicts	Linear predicts	Quadratic predicts
1	0.62	0.413	0.560	0.498
2	0.50	0.453	0.570	0.570
4	0.47	0.463	0.448	0.330
8	0.27	0.530	0.275	1.250
LOO SSE		0.1125	0.0090	1.0000

(The cubic cannot be cross-validated here: three points do not determine four parameters.)

Interpretation. The quadratic's training SSE is essentially equal to the linear model's, but its LOO error is more than 100 times larger. When it does not see the 8-day point, it bends its curvature to

fit the three short delays and predicts recall *above 1* at 8 days. The extra parameter fitted noise, not forgetting. The linear model generalizes best of the three. (None of these polynomials is a sensible *cognitive* model of forgetting, because none respects the 0–1 range; that is exactly why we use exponential and power models instead.)

1.1.6 Lab (2 hours)

Goals

1. See overfitting happen: training vs. test error as polynomial degree increases.
2. Implement item-level K-fold CV with the out-of-sample binomial log-likelihood, and compare four forgetting models.
3. Run a generalization-criterion test: train on short delays, predict long delays.
4. Interpret which model generalizes, and why the best-fitting model is not the best predicting one.

Timing: Part A 25 min · Part B 45 min · Part C 25 min · Part D 25 min.

Setup: data and four models

Code · کد: `code/week06.py`, cell 1 · سلول 1

Expected output:

```
exp      k = 2  training NLL = 29.812  params = [0.638 0.046]
pow      k = 2  training NLL = 19.830  params = [0.877 0.421]
expasym  k = 3  training NLL = 18.919  params = [0.861 0.38  0.308]
two      k = 4  training NLL = 18.919  params = [0.265 0.    0.596 0.38 ]
```

By training NLL, the more complex models win. Notice that the two-trace model's best fit has a slow component with rate 0: it has collapsed into the exponential-with-asymptote model (same NLL), as in Week 3. Now let's see whether they also *predict* better.

Part A: overfitting with polynomials (25 min)

We use polynomials in $x = \ln(1 + t)$ as deliberately "dumb" flexible models. The test data are 500 fresh datasets simulated from the true power model, with the same design.

Code · کد: `code/week06.py`, cell 2 · سلول 2

Expected output:

```
 degree  k  train_MSE  test_MSE
      0  1    0.03980   0.05072
      1  2    0.00663   0.00787
      2  3    0.00204   0.00634
      3  4    0.00202   0.00640
      4  5    0.00173   0.00661
      5  6    0.00087   0.00750
      6  7    0.00087   0.00750
      7  8    0.00000   0.00833
```

What to notice: training error decreases with every added parameter and reaches 0 at $k = 8$ (interpolation). Test error is lowest at an intermediate complexity and then rises: the extra parameters chase noise. Even the best polynomial predicts worse than the *true* power model, whose test MSE you can compute as an exercise.

Part B: K-fold cross-validation over items (45 min)

Each interval tested 50 items. We assign each item to one of $K = 5$ folds *within* each interval, so that every training set contains all 8 intervals.

> Code · کد: `code/week06.py`, cell 3 · سلول 3

Expected output (about 1–2 minutes):

```
         k  train_NLL  CV_NLL_mean  CV_NLL_sd
exp      2     29.812      256.458      1.058
pow      2     19.830      245.791      0.734
expasym  3     18.919      247.370      1.574
two      4     18.919      247.531      1.172

CV-NLL difference to the power model (mean, SD over repeats):
         exp  pow  expasym    two
mean  10.667  0.0    1.579  1.740
std    1.137  0.0    1.807  1.349
```

Note that training NLL (binomial, with the binomial coefficients) and CV-NLL (Bernoulli, item level) are on different scales. Compare models within a column, not across columns.

Interpret: 1. Which model predicts held-out items best? Is its advantage over the runner-up larger than the variability across CV repetitions? 2. The two-trace model has the best training fit. What happens to it under CV, and why? 3. Can CV tell the power and exponential-with-asymptote models apart with these data? What would help?

Part C: the generalization criterion: predicting longer delays (25 min)

A model that has captured *how* forgetting works should be able to extrapolate. Train on the five shortest delays ($t \leq 7$ days) and predict the three longest (14, 21, 30 days), which the models have never seen.

> Code · کد: `code/week06.py`, cell 4 · سلول 4

Expected output:

```
         test_NLL_long_delays  pred_t14  pred_t21  pred_t30
model
exp                    72.550     0.086     0.028     0.007
pow                     9.627     0.228     0.186     0.156
expasym                 6.268     0.260     0.257     0.257
two                    46.072     0.131     0.057     0.020
observed: {14: 0.26, 21: 0.26, 30: 0.28}
```

What to notice: here the exponential-with-asymptote model extrapolates best, *not* the true power model. The three held-out points happen to be flat (0.26, 0.26, 0.28), and a curve that levels off predicts them well. The power model keeps declining, as the true process does, but in this sample the noise at the long delays made recall look flat. The pure exponential and the two-trace model (whose short-delay fit gives no constraint on the slow component) extrapolate badly. Lessons: (1) different generalization criteria can rank models differently; (2) a single test on three data points is itself noisy, so a model's win on one test set is not proof; (3) agreement across several criteria (Part B, Part C, Week 7's AIC/BIC) is more convincing than any one of them.

Part D: put it together (25 min)

Fill in this table in your notebook from Parts B and C, and write a short paragraph (5–8 sentences) answering: Which model would you recommend as the best account of these forgetting data, and how confident are you?

Model	k	Training NLL	5-fold CV-NLL (mean $\pm$ SD)	Long-delay test NLL	Residual pattern (Week 4)	Parameters plausible?
exp						
pow						
expasym						
two						

Exercises (increasing difficulty)

Exercise 1 $\star$. In Part A, add the true power model with the true parameters as a "model" (it has no fitted parameters). Compute its test MSE and compare it with the best polynomial's. What is the irreducible error?

Exercise 2 $\star$. Rerun Part B with $K = 2$, $K = 10$ and leave-one-item-out ($K = 50$). Does the ranking change? Which K gives the most stable results across repetitions?

Exercise 3 $\star\star$ (data leakage). Write a "wrong" CV in which the starting values for each fold are taken from the fit to the *full* dataset, and another in which the fold's test items are accidentally included in training. Compare the results with the correct CV. Which mistake matters more, and why?

Exercise 4 $\star\star$ (more data). Simulate a new dataset from the true power model with $n = 500$ items per interval and rerun Parts B and C. Does the ranking become clearer? Does the two-trace model ever win?

Exercise 5 $\star\star\star$ (Advanced: leave-one-participant-out). Use the practice data generator from Week 5, Part D, with 20 participants. Implement CV in which each fold leaves out *one participant*: fit an aggregate (group-average) model on 19 participants and score the prediction of the held-out participant's curve with the Gaussian log-likelihood. Compare the exponential and power models. Then do the same with individual-level fits. Explain why the two CV schemes answer different scientific questions.

Lab check-off

Show: (1) the training-vs-test plot from Part A, (2) the CV summary table from Part B, and (3) your Part D paragraph.

1.1.7 Connection to AI & LLMs

Benchmark contamination is data leakage. LLMs are trained on huge amounts of internet text. If benchmark questions (or their answers) were published online, they may be in the training data, so the model is tested on items it has already seen. This is exactly the leakage problem of cross-validation: the test set is no longer independent of the training set, so the score measures memorization as well as generalization. It is one of the most serious problems in evaluating AI systems, including claims that LLMs show human-like cognitive abilities on classic psychology tasks, whose materials are widely published.

Code · کد: `code/week06.py`, cell 5 · سلول 5

Expected output:

```
Reported accuracy (all items): 0.73
Accuracy on leaked items:      0.96  (n = 120)
Accuracy on clean items:       0.63  (n = 280)
```

The reported score (0.73) overstates the ability to solve *new* problems (0.63). The fix is the same as in this week's lab: evaluate on items that were guaranteed not to be used in fitting, for example newly written items, or systematic variations of classic tasks.

Try it (no API needed). - ⋆ Change the share of leaked items from 0 to 0.9 and plot the reported accuracy. Can you tell from the reported accuracy alone how much leaked? - ⋆⋆ In practice we do not know which items leaked. Suppose a "fresh" variant of every item (same logic, new wording) is available. Simulate fresh variants with $P(\text{correct}) = 0.60$ and estimate the leakage effect as the difference between original and fresh accuracy, with a paired bootstrap CI (Week 5). - ⋆⋆ Discussion: an LLM solves a famous reasoning puzzle from a 1970s psychology paper. Why is this weak evidence that it "reasons like a human"? Which test would be stronger?

1.1.8 Common mistakes & misconceptions

1. "The model with the best fit is the best model." Fit rewards flexibility. Always ask how well the model predicts new data.
2. "Flexibility = number of parameters." Functional form matters too. Models with equal k can differ greatly in what they can fit.
3. Leaking test information into training, e.g., choosing starting values, bounds or transformations from the full dataset, or normalizing with full-data statistics.
4. Splitting along the wrong unit. If the claim is about new participants, holding out random trials overestimates generalization.
5. Over-reading small CV differences. CV estimates are noisy. Repeat the splits and look at the variability of the differences.
6. "CV picked the true model." CV picks the best *predictor* for this amount of data. With little data, a simpler (even wrong) model can predict better than the true one.
7. Using R^2 to compare nonlinear cognitive models. It is not comparable across models and rewards flexibility.

1.1.9 Check your understanding

1. Why does training error always decrease (or stay equal) when we add a parameter to a nested model? Answer The larger model contains the smaller one as a special case (e.g., by setting the extra parameter to 0), so its best fit can never be worse. The optimizer can always reproduce the smaller model's fit and usually improves on it by fitting noise.

2. Define overfitting in terms of signal and noise. Answer Data = signal (the systematic process) + noise (random variation). An overfitting model adapts to the noise of the particular sample. That improves its fit to these data but harms prediction of new data, in which the noise is different.

3. Describe K-fold cross-validation in four steps. Answer (1) Split the data into K folds. (2) For each fold, fit the model on the other $K - 1$ folds. (3) Score the fitted model's predictions for the held-out fold (e.g., with the log-likelihood). (4) Sum or average the scores over folds; repeat with new random splits if possible.

4. In the forgetting experiment, why do we assign items to folds *within* each retention interval rather than leaving out whole intervals? Answer Within-interval splitting tests prediction of new trials under the same conditions, and every training fold still contains every condition. Leaving out whole intervals tests a different claim: extrapolation to new conditions (the generalization criterion, Part C).

5. Model A has CV-NLL 2.1 nats lower than model B on average, but the difference varies with an SD of 3.0 across CV repetitions. What do you conclude? Answer The evidence is weak. Strictly, the SD across repetitions reflects split-to-split variability rather than the sampling uncertainty of the data, but either way a 2-nat difference with this much instability should not be over-interpreted. Collect more data or report the models as roughly equally predictive.

6. Why can a simpler, "wrong" model sometimes cross-validate better than the true, more complex model? Answer With limited data, the complex model's parameters are estimated with high variance, and that variance hurts prediction more than the simpler model's bias does (the bias–variance trade-off). With more data, the true model's variance shrinks and it eventually wins.

7. A student reports $R^2 = 0.97$ for a nonlinear forgetting model and concludes that it is correct. Give two objections. Answer (i) R^2 is high for almost any monotone decreasing curve, because most variance in forgetting data is the overall decline. (ii) A good fit says nothing about generalization or flexibility; residual patterns, CV and a comparison with alternative models are needed.

1.1.10 Assignment

A3: Model comparison and recovery (1.0 point of 20; released Week 6, due at the start of the Week 9 lecture)

A3 spans this week and next: Tasks 1–3 use this week's material, Tasks 4–6 use Week 7's (AIC/BIC, parameter and model recovery).

Data. A forgetting experiment with 12 participants, generated by this code:

Code · کد `code/week06.py`, cell 6 · سلول 6

Tasks.

1. Individual fits (1.5 pts). Fit the exponential, power and exponential-with-asymptote models to each participant by multi-start MLE. Report a table of parameters and NLLs; flag estimates on bounds.
2. Absolute fit (1 pt). Compute G^2 per participant and model. How many participants does each model fail to fit ($p < .05$)?
3. Cross-validation (2 pts). For each participant, run 5-fold item-level CV (repeated 5 times) for the three models. Report which model wins for how many participants, and the summed CV-NLL across participants with its variability.
4. Information criteria (1.5 pts, Week 7). Compute AIC, AICc and BIC for every participant and model, and the Akaike weights. Do the criteria agree with CV?
5. Recovery (2.5 pts, Week 7). (a) Parameter recovery for the power model at this design: 200 simulations with parameters drawn from plausible ranges; plot true vs. recovered values, report correlations, and comment on any trade-off. (b) Model recovery for the three models with AIC and BIC: a 3×3 confusion matrix from 100 simulations per generating model.
6. Interpretation (1.5 pts, max. 400 words). Which model best describes forgetting in this study, how strong is the evidence, and how does the recovery analysis qualify your conclusion?

Rubric (10 points, scaled to 1.0): the points per task are given above. Within each task: correctness 60%, clarity of tables and figures 20%, interpretation 20%. Late submissions lose 10% per day (max. 3 days).

1.1.11 Further reading

- Farrell & Lewandowsky (2018), chapter 10. Model comparison: why fit is not enough, flexibility, and cross-validation in relation to other comparison methods. *(Main reading.)*
- Azimifar & Ahmadi (1396), vol. 1 (in Persian): the material on generalization, overfitting and the bias–variance trade-off from the pattern-recognition perspective [verify chapter].
- *Optional classics, outside the course list:* Roberts, S., & Pashler, H. (2000). How persuasive is a good fit? A comment on theory testing. *Psychological Review, 107*, 358–367. Busemeyer, J. R., & Wang, Y.-M. (2000). Model comparisons and model selections based on generalization criterion methodology. *Journal of Mathematical Psychology, 44*, 171–189.

Preview for Week 7: cross-validation is general but computationally expensive. Information criteria (AIC, BIC) estimate generalization from a single fit. And before trusting any comparison, we must check whether our method can recover the parameters and the models that actually generated the data.